

GitHub Complete Hands-On Lab (HOL)

Introduction

Modern software development requires collaboration between developers, teams, testers, DevOps engineers, and cloud engineers. While Git helps developers manage version control locally, organizations also need a centralized platform where repositories can be stored, shared, backed up, and collaboratively managed.

This requirement led to the creation of Git hosting platforms.

GitHub is the most popular cloud-based Git repository hosting platform used worldwide.

GitHub enables developers to:

- Store repositories remotely
- Collaborate with teams
- Share source code
- Maintain centralized backup
- Manage pull requests
- Track issues
- Integrate CI/CD pipelines
- Perform code reviews
- Automate DevOps workflows

This document explains:

- What is GitHub
- Why GitHub is needed
- Difference between Git and GitHub
- GitHub architecture and workflow
- GitHub account creation process
- Remote repository concepts
- Push, Pull, and Clone concepts
- Complete Hands-On Lab with real commands
- Detailed explanation of every command

What is GitHub?

GitHub is a cloud-based platform that hosts Git repositories.

GitHub allows developers and organizations to store their Git repositories on remote servers.

GitHub is built on top of Git.

Git handles version control operations.

GitHub provides:

- Remote storage
- Collaboration features
- Web interface
- Access management
- Repository hosting
- Team workflows
- CI/CD integrations
- Issue tracking
- Pull request management

GitHub is widely used in:

- Software engineering
- DevOps
- Data engineering
- Machine learning
- Open-source development
- Enterprise cloud projects

Why We Need GitHub

Git alone works locally.

Suppose a developer stores code only on their laptop.

Problems may occur:

- Laptop crashes
- Hard disk failure
- Code gets deleted
- Team collaboration becomes difficult
- No centralized backup

GitHub solves these problems.

GitHub provides a centralized remote location where repositories are safely stored.

Teams can:

- Push code
- Pull latest changes
- Clone repositories
- Create branches
- Review pull requests
- Collaborate globally

GitHub acts like a cloud backup system for Git repositories.

Difference Between Git and GitHub

Many beginners think Git and GitHub are the same, but they are different.

Git

Git is a distributed version control system.

Git is installed locally on the machine.

Git performs:

- Version tracking
- Commit management
- Branching
- Merging
- Local repository management

Git works even without internet.

Examples:

```
git init
git add
git commit
```

GitHub

GitHub is a cloud platform that hosts Git repositories.

GitHub provides:

- Remote repository storage
- Collaboration tools
- Team management
- Pull requests
- Web UI
- CI/CD integration
- Security controls

GitHub requires internet access.

Examples:

```
git push
git pull
git clone
```

Real Difference Between Git and GitHub

Feature	Git	GitHub
Type	Version Control Tool	Cloud Hosting Platform
Installation	Installed locally	Accessed through internet
Internet Required	No	Yes
Purpose	Manage versions	Host repositories
Works Offline	Yes	No
Collaboration	Limited locally	Excellent collaboration
Maintains History	Yes	Yes
Stores Code	Local machine	Cloud servers
Developed By	Linus Torvalds	GitHub Inc

What is a Remote Repository?

A remote repository is a repository hosted on a remote server.

Example:

GitHub repository.

Developers use remote repositories to:

- Share code
- Collaborate
- Push commits
- Pull updates
- Maintain backup

Common remote names:

- origin
- upstream

The word origin usually refers to the primary GitHub repository.

Important GitHub Operations

Push

Push uploads local commits to GitHub.

Command:

```
git push
```

Pull

Pull downloads latest changes from GitHub.

Command:

```
git pull
```

Clone

Clone downloads a complete repository from GitHub.

Command:

```
git clone
```

GitHub Workflow

Typical GitHub workflow:

Local Repository → Push → GitHub Remote Repository

GitHub Remote Repository → Pull → Local Repository

Create GitHub Account

Step 1 – Open GitHub Website

Official website:

<https://github.com/>

Step 2 – Click Sign Up

Click:

Sign Up

Step 3 – Enter Details

Provide:

- Username
 - Email
 - Password
-

Step 4 – Verify Account

Complete email verification.

Step 5 – Login

Login using:

- Username
 - Password
-

Step 6 – Create Repository

Click:

New Repository

Repository Name:

test_system

Choose:

- Public or Private

Click:

Create Repository

Configure Local Repository with GitHub

Step 1 – Add Remote Repository

Command:

```
git remote add origin https://github.com/Shyamsandeep28/test_system.git
```

Explanation:

This command connects the local Git repository with the remote GitHub repository.

Breakdown:

- git remote → Manage remote repositories
- add → Add new remote
- origin → Remote repository name
- URL → GitHub repository URL

Now local repository knows where to push code.

Step 2 – Verify Git Configuration

Command:

```
git config --list
```

Explanation:

Displays:

- Username
- Email
- Repository settings
- Remote configurations

Useful for troubleshooting.

Step 3 – List Files

Command:

```
ls
```

Explanation:

Displays current files inside repository.

Step 4 – Push Local Code to GitHub

Command:

```
git push -u origin master
```

Explanation:

This uploads local commits to GitHub.

Breakdown:

- `git push` → Upload commits
- `-u` → Set upstream tracking
- `origin` → Remote repository name
- `master` → Branch name

After this:

- Local branch tracks remote branch
- Future pushes become easier

Example future push:

```
git push
```

Step 5 – Move Outside Repository

Command:

```
cd ..
```

Explanation:

Moves one directory back.

Step 6 – Create New Directory

Command:

```
mkdir test
```

Explanation:

Creates a new folder named test.

Step 7 – Move into test Directory

Command:

```
cd test/
```

Explanation:

Moves into test directory.

Step 8 – Clone GitHub Repository

Command:

```
git clone https://github.com/Shyamsandeep28/test_system.git
```

Explanation:

This downloads the complete repository from GitHub.

Git clone downloads:

- Source code
- Commit history
- Branches
- Configuration

A new folder automatically gets created.

Step 9 – Verify Cloned Repository

Command:

```
ls
```

Explanation:

Shows cloned repository folder.

Step 10 – Enter Repository

Command:

```
cd test_system/
```

Explanation:

Moves inside cloned repository.

Step 11 – Verify Repository Files

Command:

```
ls
```

Explanation:

Displays repository files.

Step 12 – Check Commit History

Command:

```
git log
```

Explanation:

Displays:

- Commit history
- Commit IDs
- Author information
- Commit messages
- Timestamps

Useful for auditing and rollback.

Step 13 – Verify Files Again

Command:

```
ls
```

Explanation:

Displays files again.

Step 14 – Move Back One Directory

Command:

```
cd ..
```

Explanation:

Moves one level back.

Step 15 – Pull Repository Using URL

Command:

```
git pull https://github.com/Shyamsandeep28/test_system.git
```

Explanation:

This pulls updates directly from repository URL.

However, this is not the common practice.

Normally developers use remote names.

Step 16 – Pull Using Remote Name

Command:

```
git pull origin
```

Explanation:

Downloads latest changes from origin remote.

Git automatically identifies the current branch.

Step 17 – Verify Files

Command:

```
ls
```

Explanation:

Displays updated files.

Step 18 – Pull Latest Changes

Command:

```
git pull
```

Explanation:

Downloads latest changes from tracked remote branch.

This is the most common pull command.

Step 19 – Pull Specific Branch

Command:

```
git pull origin master
```

Explanation:

Downloads changes specifically from:

- Remote: origin
- Branch: master

Useful in multi-branch environments.

Step 20 – Verify Repository Status

Command:

```
git status
```

Explanation:

Displays:

- Modified files
 - Staged files
 - Current branch
 - Synchronization status
-

Step 21 – Move into Repository

Command:

```
cd test_system/
```

Explanation:

Moves into repository.

Step 22 – Pull Latest Changes Again

Command:

```
git pull origin master
```

Explanation:

Synchronizes local repository with GitHub.

Step 23 – Verify Repository Files

Command:

```
ls
```

Explanation:

Displays latest repository files.

Step 24 – View Command History

Command:

```
history
```

Explanation:

Displays previously executed terminal commands.

Useful for:

- Troubleshooting
 - Auditing
 - Repeating commands
 - Learning workflows
-

Understanding Push, Pull, and Clone

Clone

Clone is used only once initially.

Purpose:

Download complete repository.

Push

Push uploads local commits to GitHub.

Used after local development.

Pull

Pull downloads latest updates from GitHub.

Used before starting development.

Real-Time Enterprise Workflow

Suppose an enterprise has multiple developers.

Workflow:

1. Developer clones repository
2. Developer creates feature branch
3. Developer modifies code
4. Developer commits changes
5. Developer pushes branch
6. Pull request gets created
7. Team reviews code
8. Code merges into main branch
9. Other developers pull latest changes

This enables large-scale collaboration safely.

Best Practices for GitHub

- Always pull latest changes before push
- Use meaningful commit messages
- Never push sensitive credentials
- Use branches for new features
- Review pull requests carefully
- Keep repositories organized
- Use .gitignore properly
- Regularly synchronize repositories

Common GitHub Errors

Authentication Failed

Occurs due to:

- Wrong credentials
- Expired token
- Incorrect repository permissions

Merge Conflict

Occurs when multiple developers modify same lines.

Git cannot automatically decide which change to keep.

Repository Not Found

Occurs due to:

- Wrong repository URL
- Repository deleted
- No access permission

Permission Denied

Occurs due to:

- SSH key issue
- Access restrictions
- Incorrect auth